Food Orders Dataset: Data Understanding, Transformation & Validation

Student practical assignment | Dataset: food_orders.csv

Business scenario

You are a junior data analyst/data engineer at an online food-delivery company. The operations team has shared a CSV file containing order, customer, restaurant, payment, delivery, refund, weather, and order-channel information. Before the company can use this data for reporting, you must understand its structure, identify data-quality issues, clean and transform it, validate the results, and summarize useful business insights.

Your goal

Complete this assignment entirely in this Jupyter Notebook (.ipynb). Use Python and Pandas to:

1. Understand the dataset and its columns.
2. Profile missing values, duplicates, data types, and invalid records.
3. Decide and document suitable data-cleaning rules.
4. Transform the data and create useful derived columns.
5. Validate the transformed dataset.
6. Answer the business questions and save the cleaned dataset.

Working rules

- Keep the original data in a variable named raw_df and perform work on a separate copy such as work_df.
- Add short Markdown explanations before or after important code cells.
- Do not replace every missing value with zero or the mean without justification.
- A missing refund is not automatically a zero refund; a missing offer is not automatically “No Discount.”
- Document assumptions and report any missing values that you intentionally retain.
- Run all cells from top to bottom before submitting.

Task 1 — Load and understand the dataset

Questions / activities

1. Import the Python libraries required for data loading and analysis.
2. Load food_orders.csv into raw_df.
3. Display the first five rows, last five rows, and a random sample of five rows.
4. Display the number of rows and columns, column names, and data types.
5. Use suitable summary methods to understand numeric columns and categorical columns.
6. For each column, describe what information it appears to contain and identify its expected data type (identifier, datetime, numeric, or category).

Write a short Markdown observation: What does one row represent, and which columns may be useful for analyzing order value, delivery, payments, or platform revenue?

Task 2 — Perform a data-quality assessment

Use code to answer each question.

1. How many missing values are present in each column?
2. What percentage of each column is missing?
3. How many fully duplicated rows exist?
4. Are there repeated Order ID values? Check before deciding what to remove.
5. Which columns have unexpected data types?
6. Can the order and delivery timestamps be parsed as datetime values?
7. Are there records where delivery occurs before the order?
8. Are there negative or otherwise suspicious values in financial columns or calculated delivery duration?
9. Check the distinct values and frequency counts for Payment Method, Discounts and Offers, Weather Condition, and Order Channel.

Deliverable: Add a Markdown summary listing at least three data-quality issues you found and why each matters to the business.

Task 3 — Design your cleaning rules before changing data

Create a small Markdown table in this notebook with these headings:

Data-quality issue	Your chosen handling rule	Reason / business assumption

Include rules for:

- Exact duplicate rows and repeated order identifiers.
- Missing Order Value.
- Missing Delivery Fee, Commission Fee, Payment Processing Fee, and Refunds/Chargebacks.
- Missing Payment Method and Discounts and Offers.
- Invalid timestamps and impossible delivery durations.

Important: Do not automatically treat missing financial values as zero. For example, exclude records with missing order value from order-value calculations, or choose another defensible method and explain it. Categorical values such as payment method may use an Unknown label. Missing offer details may use Not Specified, which does not claim that no offer was applied.

Task 4 — Clean the dataset

Create work_df as a copy of raw_df; do not modify raw_df directly.

1. Remove fully duplicated rows only after inspecting and documenting them.
2. Check repeated Order ID values and decide whether they are true duplicate transactions or require investigation.
3. Convert Order Date and Time and Delivery Date and Time to datetime data types, handling parsing errors explicitly.
4. Convert financial columns to appropriate numeric types where necessary.
5. Apply the cleaning rules you documented in Task 3.
6. Keep track of how many rows were removed or retained and why.
7. Display the first five rows of the cleaned data and its data types.

Questions: How many rows remain after cleaning? Which missing values remain, and why is it acceptable or necessary to retain them?

Task 5 — Transform the data and create derived columns

Using the cleaned data, create the following columns:

1. **Delivery_Time_Minutes** — difference between Delivery Date and Time and Order Date and Time, expressed in minutes.
2. **Order_Date** — date portion of the order timestamp.
3. **Order_Hour** — hour of the day when the order was placed.
4. **Platform_Net_Revenue** — use this simplified exercise formula:

Commission Fee + Delivery Fee - Payment Processing Fee - Refunds/Chargebacks

Only calculate this metric when all four component values are available. Otherwise, keep the result missing and explain the limitation. This is a simplified analytical measure for the assignment, not a complete accounting definition.

Questions:

- Which records cannot have a delivery duration calculated, and why?
- Are any calculated delivery durations negative or implausible? Flag and investigate them instead of silently accepting them.
- Why should missing values in the revenue formula not automatically be replaced with zero?

Task 6 — Answer the business questions

Use Pandas calculations and/or groupby() to answer the questions below. Display the result clearly and add a short interpretation in Markdown. Do not guess or hard-code answers.

1. How many records were present in the original dataset and how many remain in the cleaned dataset?
2. How many records have a valid Order Value?
3. What are the total, mean, and median order values, calculated using valid order-value records?
4. Which payment method has the largest number of orders with a known payment method?
5. Which order channel has the most orders?
6. Which weather condition appears most frequently?
7. What are the mean and median delivery times for records with valid delivery durations?
8. Which five restaurants have the highest order counts?

9. What is the total Platform_Net_Revenue across records where the required components are available? State how many records were included in this calculation.

For each financial metric, clearly state which records were included and whether missing data affects the result.

Task 7 — Create at least two visualizations

Create at least **two readable charts** that help the business understand the data. Suggested choices:

- Number of orders by Order Channel.
- Number of orders by Payment Method.
- Average delivery time by Weather Condition.
- Daily order count using Order_Date.

Give each chart a title and axis labels where applicable. Add one or two sentences explaining what the chart suggests. Do not present a relationship as causal unless the data supports that conclusion.

Task 8 — Validate the transformed dataset

Write code to check the following and show the results:

1. The cleaned dataset has the expected columns.
2. The datetime and financial columns have appropriate data types.
3. Fully duplicated rows have been handled according to your documented rules.
4. Repeated Order ID values have been checked and explained.
5. No negative delivery durations remain in the records you classify as valid; any flagged records are documented.
6. The derived delivery-time calculation is correct for a few manually inspected records.
7. The revenue formula is correct for a few records with all required values present.
8. Missing-value counts before and after cleaning are compared.
9. The number of records removed, retained, and excluded from key calculations is reported.

Validation principle: The goal is not necessarily to make every missing-value count zero. The goal is to ensure that remaining issues are understood, documented, and handled consistently.

Data Engineering Task:

Task: Build a Batch-Based ELT-Style Pipeline Using Python, Pandas and MySQL

Scenario

You are working as a Junior Data Engineer for an online food delivery company. The company provides a food_orders.csv dataset containing food order information.

Your responsibility is to develop a batch-based data pipeline that first loads the original data into a MySQL database, retrieves the raw data, transforms it using Pandas, validates the results, and loads the processed data into a separate MySQL database.

Requirements

1. Extract the Source Data

- **Read food_orders.csv using Pandas.**
- **Extract records in batches of 100 rows.**
- **Display the batch number and record count.**
- **Do not clean, remove duplicates, or transform the data during the initial extraction.**

2. Load Raw Data into MySQL

- **Create a MySQL database named food_orders_raw_db.**
- **Create a table named raw_food_orders.**
- **Load the original records into the raw table in batches of 100 rows.**
- **Preserve the original values, missing values, and duplicate records.**
- **Ensure that loading one batch does not overwrite previously loaded batches.**
- **Use database transactions for batch insertion.**
- **Verify the total number of records loaded into the raw table.**

3. Retrieve Raw Data from MySQL

- **Connect to food_orders_raw_db using Python.**
- **Retrieve the data from raw_food_orders in batches of 100 rows.**
- **Load each retrieved batch into a Pandas DataFrame.**
- **Display the batch number and number of records retrieved.**

- **Do not modify the original raw table.**

4. Transform Data Using Pandas

For each retrieved batch:

- **Remove duplicate records using appropriate duplicate identification rules.**
- **Handle missing values using suitable business rules.**
- **Convert order and delivery timestamps into appropriate datetime formats.**
- **Convert financial columns into appropriate numeric data types.**
- **Create a Delivery_Time_Minutes column using the order and delivery timestamps.**
- **Create a Platform_Net_Revenue column using the following formula:**

Platform Net Revenue = Commission Fee + Delivery Fee − Payment Processing Fee − Refunds/Chargebacks

- **Identify invalid records, such as records with invalid timestamps or negative delivery durations.**
- **Document the transformation and missing-value handling decisions.**

Ensure that your duplicate-handling strategy also considers duplicate records that may occur across different batches.

5. Validate the Transformed Data

- **Check for remaining missing values.**
- **Check for duplicate records.**
- **Verify the data types of the transformed columns.**
- **Validate delivery duration calculations.**
- **Validate platform net revenue calculations.**
- **Separate or record rejected records and their rejection reasons.**
- **Generate a validation report for the processed data.**

6. Load Transformed Data into a Separate MySQL Database

- **Create another MySQL database named food_orders_analytics_db.**
- **Create a table named cleaned_food_orders.**
- **Define appropriate column data types.**

- **Load the validated, transformed records into the table in batches of 100 rows.**
- **Use batch insertion instead of inserting records individually.**
- **Ensure that processing a new batch does not overwrite previously loaded batches.**
- **Handle database insertion failures using transactions and meaningful error messages.**
- **Verify that the final database table contains the expected processed records.**

7. Pipeline Execution

Organize the project into the following Python files:

- **config.py — Store file paths, batch size, and database configuration.**
- **extract.py — Read the source CSV in batches.**
- **load_raw.py — Load original data into the raw MySQL database.**
- **retrieve_raw.py — Retrieve raw records from MySQL in batches.**
- **transform.py — Clean and transform the retrieved DataFrames using Pandas.**
- **validate.py — Validate the transformed data and report rejected records.**
- **load_transformed.py — Load processed records into the analytics database.**
- **elt_batch_pipeline.py — Execute the complete workflow in the correct order.**

8. Business Analysis Using SQL

Using the cleaned_food_orders table, write SQL queries to answer:

1. **What is the total order value?**
2. **What is the average order value?**
3. **Which payment method has the highest number of orders?**
4. **Which order channel has the highest number of orders?**
5. **What is the average delivery time in minutes?**
6. **What is the total platform net revenue according to the specified formula?**

9. Final Execution Report

Display the following information after pipeline execution:

- **Total records extracted from the CSV.**
- **Total number of batches processed.**
- **Total raw records loaded into MySQL.**
- **Total raw records retrieved from MySQL.**
- **Total valid records transformed.**
- **Total rejected records.**
- **Total transformed records loaded into the analytics database.**
- **Number of successful and failed batches.**
- **Final validation status.**

Expected Output

A working batch-based Python pipeline that preserves the original food order data in food_orders_raw_db, retrieves the data from MySQL, transforms and validates it using Pandas, and loads the processed results into food_orders_analytics_db.

Submission Requirements

Submit the complete Python project folder, requirements.txt, README.md, and the generated validation report.

The README must explain the project structure, database setup, configuration, execution commands, and sample results.